

WHATSAPP REMEDIATION REPORT

1. Implementation Summary

- The WhatsApp integration for the **AL-AZHARI LIBRARY backend** has been brought to full compliance with **Section 38** (38.1 MVP , 38.2 Contextual actions , 38.3 Future adapter) and **PRD Section 6.24** (WA-001-003).
- **Safe URL Generator (buildWhatsAppUrl)**: Generates valid `https://wa.me/<phone>?text=<encoded-message>` URLs with the official Arabic library greeting and URL-encoded contextual prefills.
 - **Phone Normalization & Validation**: Reuses canonical phone normalization (`canonicalizePhone` from `src/modules/users/utis/phone.util.ts`), strips the leading `+` to conform strictly to WhatsApp's `wa.me` specification, and rejects malformed or malicious inputs.
 - **Strict Context Allowlist**: Limits context strictly to `{ product?: string; orderReference?: string; serviceReference?: string }`. All sensitive fields (payment proofs, addresses, tokens, private notes, passwords, internal audit data) are excluded.
 - **URL Injection Defense**: Encodes all special characters (`&` , `?` , `#` , `=` , `%` , `\n` , `"` , `'` , and Arabic punctuation), ensuring the resulting URL is syntactically valid and the prefilled message remains a single `text` query parameter without parameter pollution or hash-fragment hijacking.
 - **Future Adapter (whatsappLinkAdapter)**: Implemented `whatsappLinkService` implementing `whatsappLinkAdapter` , enabling future addition of the WhatsApp Business API without modifying order or service domain logic.
 - **User-Facing Disclaimer (WA-003)**: Accompanies generated link responses with an explicit disclaimer in Arabic and English confirming that WhatsApp communication is for support only and does not create, modify, confirm, or cancel any business or payment record.
 - **Zero Authoritative State Mutation**: Verified that WhatsApp integration is purely a communication action and never mutates MongoDB business records.
 - **Open Decision (OD-01)**: The production support phone number is not hard-coded; it remains configurable via `env.WHATSAPP_PHONE` . When unconfigured, the service blocks safely with a domain error without inventing a placeholder number.
 - **API Endpoints**: Exposed public customer support link endpoint (`GET /api/v1/whatsapp/support` and `/link`) and authorized Admin customer-contact endpoint (`POST /api/v1/admin/whatsapp/customer-link` , requiring `orders.read` permission).

2. Files Changed

Created

- `src/integrations/whatsapp/whatsapp.types.ts` — WhatsApp types, DTOs, and `whatsappLinkAdapter` interface.
- `src/integrations/whatsapp/whatsapp.builder.ts` — `buildWhatsAppUrl` , `normalizeWhatsAppPhone` , constants, and URL-encoding logic.
- `src/integrations/whatsapp/whatsapp.service.ts` — `whatsappLinkService` implementing `whatsappLinkAdapter` with OD-01 handling.
- `src/integrations/whatsapp/whatsapp.schema.ts` — Zod validation schemas for query parameters and body payloads.
- `src/integrations/whatsapp/whatsapp.controller.ts` — HTTP controllers for customer and admin WhatsApp link actions.
- `src/integrations/whatsapp/whatsapp.routes.ts` — Express routers with authentication and RBAC guards.
- `src/integrations/whatsapp/index.ts` — Integration barrel export.
- `tests/unit/whatsapp-builder.test.ts` — Unit tests (phone normalization, builder, greeting, context, adapter, OD-01).
- `tests/security/whatsapp-security.test.ts` — Security tests (allowlist filtering, secret exclusion, URL injection, parameter pollution).
- `tests/api/whatsapp-api.test.ts` — API integration tests (customer support, admin customer link, RBAC, zero mutation).

Modified

- `src/integrations/index.ts` — Re-exported WhatsApp integration module.
- `src/app.ts` — Mounted `/whatsapp` on `apiRouter` and `/whatsapp` on `adminRouter`.
- `src/config/openapi.ts` — Documented WhatsApp endpoints and request/response schemas.

3. Helper / Adapter

- **File:** `src/integrations/whatsapp/whatsapp.builder.ts`
 - `buildWhatsAppUrl(phone: string, context?: WhatsAppContext): string`
 - `normalizeWhatsAppPhone(rawPhone: string): string`
 - `WHATSAPP_BASE_GREETING = 'السلام عليكم، أحتاج مساعدة من مكتبة الأزهرى'`
 - `WHATSAPP_LABEL_PRODUCT = 'المنتج: '`
 - `WHATSAPP_LABEL_ORDER = 'رقم الطلب: '`
 - `WHATSAPP_LABEL_SERVICE = 'رقم الخدمة: '`
 - `WHATSAPP_DISCLAIMER_AR`
 - `WHATSAPP_DISCLAIMER_EN`
- **File:** `src/integrations/whatsapp/whatsapp.service.ts`
 - `export interface WhatsAppLinkAdapter`
 - `export class WhatsAppLinkService implements WhatsAppLinkAdapter`
 - `export const whatsappService = new WhatsAppLinkService();`

4. Context Fields

The context object is restricted to the following three public references:

1. `product?: string` — Prefilled as `المنتج: <product>`
2. `orderReference?: string` — Prefilled as `رقم الطلب: <orderReference>`
3. `serviceReference?: string` — Prefilled as `رقم الخدمة: <serviceReference>`

5. Security Verification

All sensitive, private, or state-related fields are strictly excluded and never serialized:

- **Payment proofs:** EXCLUDED (verified in `tests/security/whatsapp-security.test.ts`)
- **Customer addresses:** EXCLUDED
- **Tokens / JWTs:** EXCLUDED
- **Session cookies:** EXCLUDED
- **Private / internal notes:** EXCLUDED
- **Cloudinary signed URLs:** EXCLUDED
- **Passwords / credentials:** EXCLUDED
- **Internal audit data:** EXCLUDED

6. URL Encoding Tests

All URL encoding and injection scenarios were tested and verified in `tests/security/whatsapp-security.test.ts` and `tests/unit/whatsapp-builder.test.ts`:

1. **Ampersand (&)**: `ORD-123&admin=true` is percent-encoded (`%26`); does not create a secondary query parameter.
2. **Question mark (?)**: `SRV-100?bypass=1` is percent-encoded (`%3F`); does not create a secondary query string.
3. **Hash fragment (#)**: `Book#frag` is percent-encoded (`%23`); hash fragment remains empty (`url.hash === ''`).
4. **Equals sign (=)**: Safely encoded as part of value.
5. **Percent sign (%)**: Safely encoded without malformed percent escape sequences.
6. **Newlines (\n)**: Encoded as `%0A` separating lines cleanly.
7. **Quotes (" , ')**: Safely encoded within text payload.
8. **Arabic punctuation (, , ؟ , :)**: Safely encoded within text payload.
9. **Phone injection**: Path traversal (`../../../../etc/passwd`), protocol schemes (`javascript: , http: ,`), and query strings rejected via validation.

7. Environment

- **Variable**: `WHATSAPP_PHONE`
- **Status**: Reused existing configuration in `src/config/env.ts`.
- **Open Decision Dependency**: `OD-01` remains open. No production phone number was fabricated or hard-coded into the repository. The implementation reads from the environment, and if `WHATSAPP_PHONE` is not configured, support URL generation safely throws a `BusinessRuleViolationError` (HTTP 422) indicating the unresolved open decision.

8. Tests

- **TypeScript Compilation (`tsc --noEmit`)**: PASS (0 errors)
- **ESLint (`eslint .`)**: PASS (0 errors, 0 warnings)
- **Build (`npm run build`)**: PASS (clean emission to `dist/`)
- **WhatsApp Test Suites**: 3 / 3 test suites passed (38 / 38 tests passed)
 - `tests/unit/whatsapp-builder.test.ts` : 17 passed
 - `tests/security/whatsapp-security.test.ts` : 7 passed
 - `tests/api/whatsapp-api.test.ts` : 14 passed
- **Full Test Suite**: 143 / 143 test suites passed (1,025 / 1,025 tests passed)

9. Regression

- **Status**: No regressions. All 140 previously passing test suites (987 tests) continue to pass with 100% green status alongside the 3 new WhatsApp test suites.

10. Out-of-Scope Findings

- **Pre-existing Concurrency Write Conflict**: `tests/integration/return-concurrency.test.ts` may intermittently encounter raw MongoDB `writeConflict` (code 112) if run under unbounded parallel worker concurrency (`--maxWorkers=16`), but passes reliably when run with `--maxWorkers=4` . Per the scoping rule, this pre-existing Phase 12 test was not modified.

11. Open Decision Dependency

- `OD-01` – Final website phone number : OPEN

- **Reason:** The official plan identifies `OD-01` as an open decision pending store owner confirmation of the final business phone number.
- **Implementation Impact:** The backend implementation relies strictly on `env.WHATSAPP_PHONE`. When configured in production deployment, it immediately activates without code modification. If left empty, support URL requests return a clear domain error referencing `OD-01`.

12. Acceptance Matrix

Requirement	Description	Status	Evidence
WA-001	Persistent / contextual customer WhatsApp support action	PASS	<code>src/integrations/whatsapp/whatsapp.controller.ts</code> (<code>getSupportLink</code>), <code>src/integrations/whatsapp/whatsapp.routes.ts</code> (<code>GET /whatsapp/support</code>), <code>tests/api/whatsapp-api.test.ts</code>
WA-002	Admin interface action to contact customer on WhatsApp	PASS	<code>src/integrations/whatsapp/whatsapp.controller.ts</code> (<code>getCustomerLink</code>), <code>src/integrations/whatsapp/whatsapp.routes.ts</code> (<code>POST /admin/whatsapp/customer-link</code>), <code>tests/api/whatsapp-api.test.ts</code>
WA-003	WhatsApp does not mutate authoritative business state	PASS	Verified zero database mutations in <code>tests/api/whatsapp-api.test.ts</code> and disclaimers in <code>src/integrations/whatsapp/whatsapp.builder.ts</code>

13. Final Status

WHATSAPP — COMPLIANT WITH OPEN DECISION DEPENDENCY